

6.性能优化实战：减少回流与重绘

一、理解回流 (Reflow) 与重绘 (Repaint)

为了优化，我们首先要精确理解这两个概念：

- **回流 (Reflow):** 当元素的尺寸、结构或位置发生改变时，浏览器需要重新计算元素的几何属性（即重新进行“布局”阶段），这个过程称为“回流”。回流会影响其周围甚至整个页面的其他节点，是开销非常大的一种操作。**回流必然导致重绘。**
- **重绘 (Repaint):** 当元素的视觉样式（如 `color` , `background-color`）发生改变，但其几何属性没有变化时，浏览器会跳过布局阶段，直接为该元素重新绘制（即重新进行“绘制”阶段），这个过程称为“重绘”。相比回流，重绘的开销较小。

优化的核心思想：避免不必要的回流，将多次回流合并为一次。

二、常见触发回流与重绘的操作

了解触发时机是避免它们的第一步。

常见触发回流 (Reflow) 的操作：

1. **页面首次渲染：**这是不可避免的回流。
2. **DOM节点操作：**增加、删除或修改DOM节点。
3. **元素几何属性变更：**修改 `width` , `height` , `padding` , `margin` , `border` 等。
4. **元素位置变更：** `position` , `top` , `left` 等。
5. **字体变更：**修改 `font-size` , `font-family` 等。
6. **窗口尺寸变更：** `resize` 事件触发。
7. **获取特定的布局属性：**这是最容易被忽略的一点！当你用JavaScript读取以下属性时，浏览器为了返回精确的值，会强制立即执行一次回流：
 - `offsetTop` , `offsetLeft` , `offsetWidth` , `offsetHeight`

- `scrollTop`, `scrollLeft`, `scrollWidth`, `scrollHeight`
- `clientTop`, `clientLeft`, `clientWidth`, `clientHeight`
- `getComputedStyle()`

仅触发重绘 (Repaint) 的操作:

- 修改 `color`, `background-color`, `visibility`, `outline`, `box-shadow` 等不影响布局的样式。

三、优化实战：具体的编码技巧

1. 批量修改DOM

问题：当需要向DOM中添加多个元素时，如果直接在循环中逐个 `appendChild`，每次操作都会触发一次回流。

代码块

```
1 // 反例：会触发多次回流
2 const container = document.getElementById('container');
3 for (let i = 0; i < 100; i++) {
4   const el = document.createElement('p');
5   el.textContent = i;
6   container.appendChild(el); // 每次循环都可能触发回流
7 }
```

优化技巧：

- 使用 `DocumentFragment`：`DocumentFragment` 是一个存在于内存中的轻量级DOM容器，对它的所有操作都不会触发回流。我们可以先把所有新节点添加到 `Fragment` 中，最后再将 `Fragment` 一次性添加到真实DOM中。

代码块

```
1 // 优例：只触发一次回流
2 const container = document.getElementById('container');
3 const fragment = document.createDocumentFragment(); // 创建文档片段
4
5 for (let i = 0; i < 100; i++) {
```

```

6     const el = document.createElement('p');
7     el.textContent = i;
8     fragment.appendChild(el); // 在内存中操作，不触发回流
9 }
10
11 container.appendChild(fragment); // 一次性插入，触发单次回流

```

- **隐藏元素后操作：**将需要多次操作的元素先 `display: none`，完成所有操作后，再恢复显示。这会产生两次回流（隐藏和显示），但远优于N次。

代码块

```

1     const container = document.getElementById('container');
2     container.style.display = 'none'; // 回流 1
3     // ... 在此进行多次DOM操作 ...
4     container.style.display = 'block'; // 回流 2

```

2. 批量修改样式与读写分离

问题：在循环中交替读取元素的布局属性和设置样式，会导致“布局抖动”（Layout Thrashing）。浏览器被迫在每次循环中都强制回流。

代码块

```

1 // 反例：布局抖动
2 const elements = document.querySelectorAll('.box');
3 for (let i = 0; i < elements.length; i++) {
4     // 读取 offsetWidth (读操作)，强制浏览器回流
5     const width = elements[i].offsetWidth;
6     // 设置 style.width (写操作)，再次导致布局变更
7     elements[i].style.width = width + 10 + 'px';
8 }

```

优化技巧：读写分离 (Read/Write Separation)

将读操作和写操作完全分开，先批量读取所有需要的值并缓存，然后批量进行样式更新。

代码块

```

1 // 优例：读写分离
2 const elements = document.querySelectorAll('.box');

```

```
3  const widths = [];  
4  
5  // 读操作  
6  for (let i = 0; i < elements.length; i++) {  
7    widths.push(elements[i].offsetWidth);  
8  }  
9  
10 // 写操作  
11 for (let i = 0; i < elements.length; i++) {  
12   elements[i].style.width = widths[i] + 10 + 'px';  
13 }
```

优化技巧：使用 `classList`

对于复杂的样式变更，推荐将样式组合定义在一个CSS类中，然后通过JavaScript切换 `class`。浏览器可以更高效地处理 `class` 的变更。

代码块

```
1  /* style.css */  
2  .large-box {  
3    width: 200px;  
4    height: 200px;  
5    border: 2px solid blue;  
6  }
```

代码块

```
1  // JS  
2  const box = document.getElementById('my-box');  
3  // 替代 style.width, style.height 等多次修改  
4  box.classList.add('large-box'); // 一次操作，浏览器会优化渲染
```

通过以上方法，我们可以有效减少渲染引擎的工作量，让网页动画和交互变得更加流畅。